1. Giới thiệu và challenge

Challenge Scenario

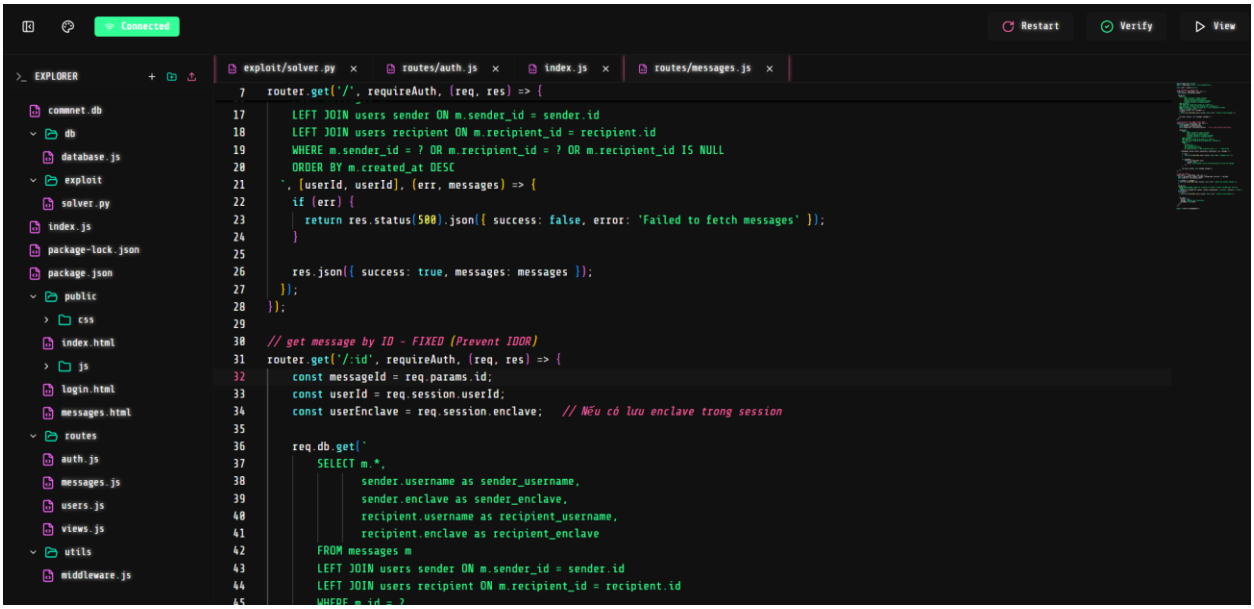
Cut off from each other and besieged by undead propaganda, humanity's survivors rely on CommNet—until the white-hats break in to silence the broadcast and reconnect the enclaves.

Challenge Scenario: Blackouts spread under the NecroNet's command, and in the flicker of dying transformers, the white-hats race to reboot the power grid before darkness becomes permanent.

Đây là một challenge kiểu **"fix the code"**. Nhiệm vụ của chúng ta không phải khai thác lỗ hổng, mà là tìm ra điểm yếu trong source code và sửa nó để ứng dụng trở nên an toàn hơn. Khi vào challenge, ta được làm việc trong giao diện code editor đặc biệt, có thể chỉnh sửa code, restart application và Verify để kiểm tra xem đã vá đúng chưa.

2. Khám phá source code ban đầu

Sau khi spawn instance và quan sát giao diện web, mình chuyển sang làm việc trong HTB Code Editor.



Khi mở Code Editor, ta thấy project được tổ chức khá rõ ràng với các thư mục chính:

- routes/ : Chứa các route xử lý API (auth.js, messages.js, users.js)
- db/ : Chứa file database và logic kết nối (database.js)
- exploit/ : Chứa file solver.py – đây là file quan trọng nhất để hiểu cách hệ thống test challenge

- public/ : Chứa các file frontend (html, css, js)
- utils/ : Chứa middleware và các hàm hỗ trợ

Mình mở file exploit/solver.py trước tiên vì đây là “chìa khóa” để hiểu lỗ hổng.

```python
def register_user(username, password):
    }

    response = post(url, json=data, allow_redirects=False)
    return response

def login_user(username, password):

    url = f"{BASE_URL}/api/auth/login"
    data = {
        "username": username,
        "password": password
    }

    response = post(url, json=data, allow_redirects=False)
    return response


def get_messages(cookie, id):
    url = f"{BASE_URL}/api/messages/{id}"
    response = get(url, cookies=cookie)
    return response


if __name__ == "__main__":
    username, password = random_string(8), random_string(8)

    register_user(username, password)
    cookies = login_user(username, password).cookies
```

Solver.py thực hiện các bước sau:

1. Tạo một user ngẫu nhiên và đăng ký (/api/auth/register)
2. Đăng nhập để lấy cookie session
3. Truy cập trực tiếp endpoint /api/messages/3

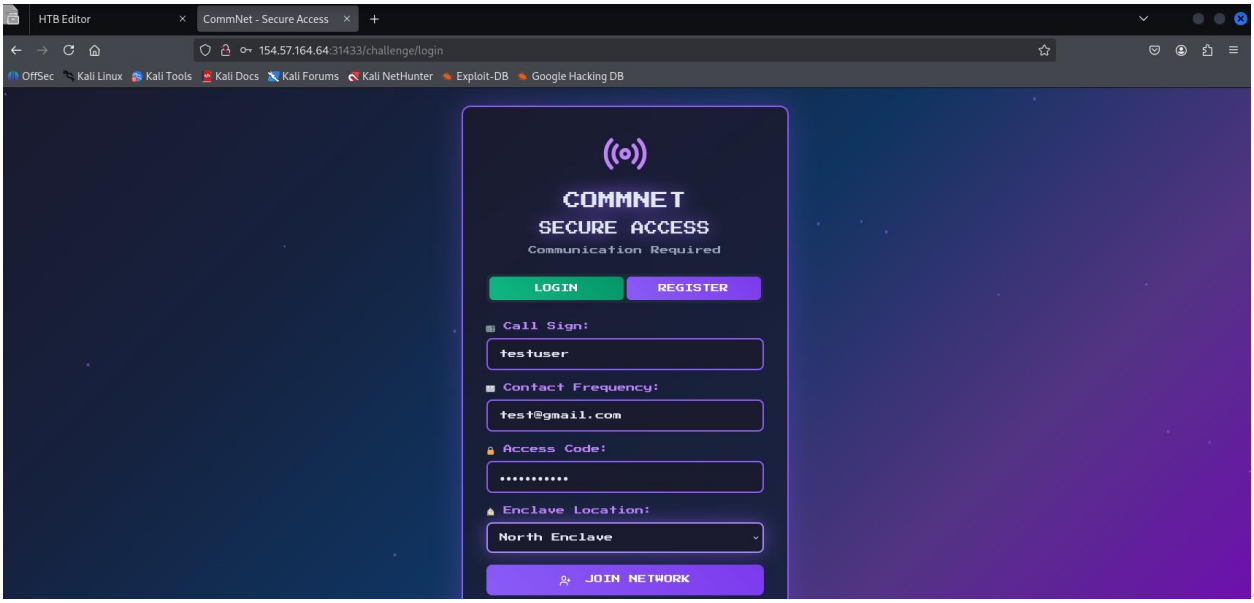
Phần quan trọng nhất trong solver.py là comment:

```python
# flag sample is on messageId 3
response = get_messages(cookies, 3)
```

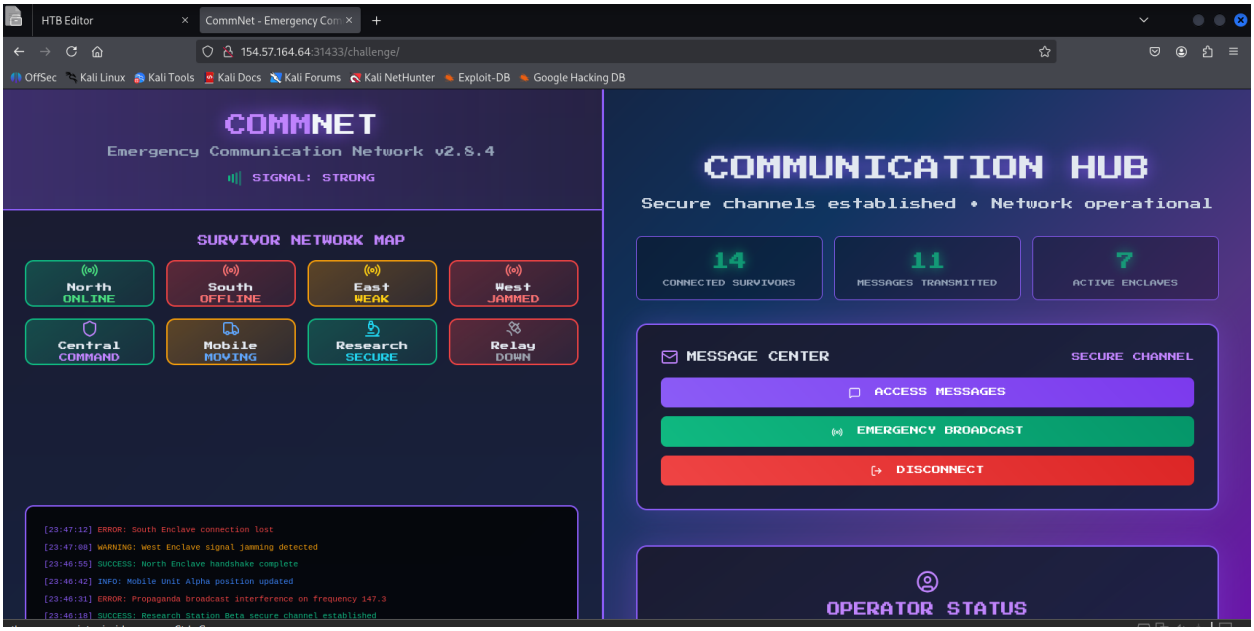
Điều này cho thấy hệ thống đã cố tình đặt flag ở message có ID = 3, và solver chỉ cần login rồi đọc thẳng message này.

Để xác nhận lỗ hổng, mình thực hiện theo đúng luồng của solver.py trên giao diện web:

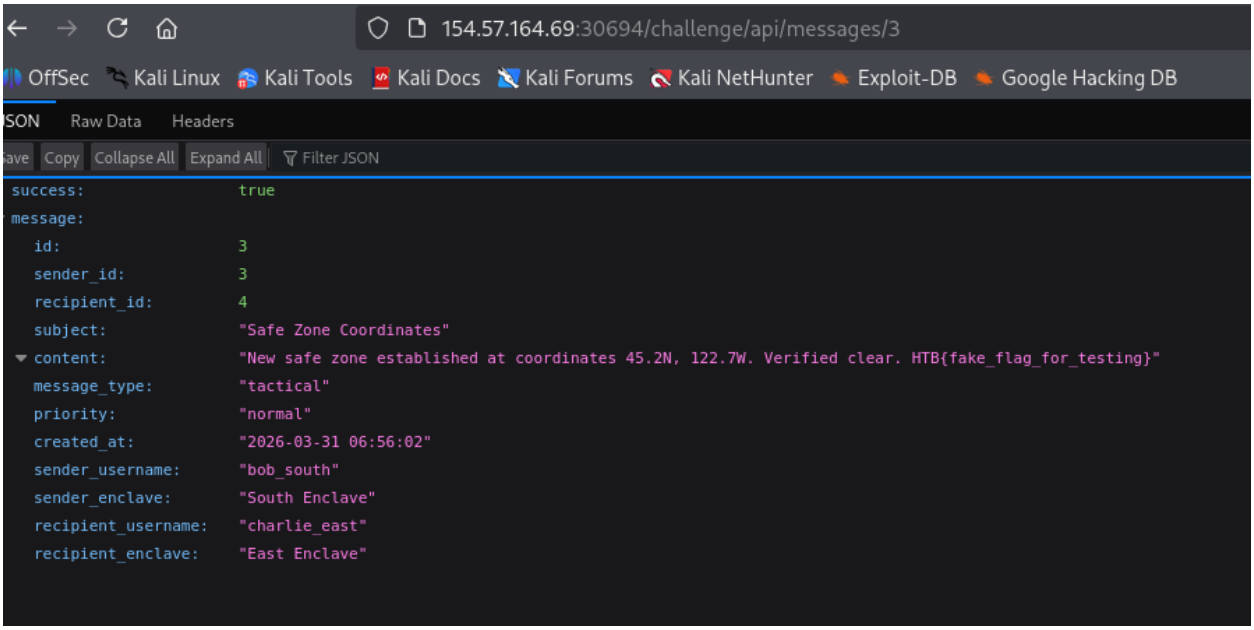
- Register một user mới với **Call Sign**: testuser và **Access Code**: password123
- Login bằng tài khoản vừa tạo
- Sau khi login thành công, thay đổi URL thành: http://154.57.164.64:31433/api/messages/3



Nhấn “Join Network”.



Ta thử đổi URL bằng cách thêm đường dẫn “/api/messages/3”.



Kết quả cho thấy: User bình thường (testuser) **vẫn đọc được đầy đủ nội dung message ID 3**, dù message này không thuộc về tài khoản của mình.

Điều này chứng minh rõ ràng tồn tại lỗ hổng **IDOR (Insecure Direct Object Reference)** tại endpoint /api/messages/:id.

3. Phân tích và tìm ra lỗ hỏng

Sau khi đọc solver.py và thực hiện test thực tế trên giao diện web, mình chuyển sang phân tích source code để tìm nguyên nhân gốc rễ của lỗ hổng.

Mình tập trung vào file **routes/messages.js**, cụ thể là hàm xử lý lấy message theo ID.

Đoạn code vulnerable như sau:

```
// get message by ID
router.get('/:id', requireAuth, (req, res) => {
  const messageId = req.params.id;

  req.db.get(`
    SELECT m.*,
           sender.username as sender_username,
           sender.enclave as sender_enclave,
           recipient.username as recipient_username,
           recipient.enclave as recipient_enclave
    FROM messages m
    LEFT JOIN users sender ON m.sender_id = sender.id
    LEFT JOIN users recipient ON m.recipient_id = recipient.id
    WHERE m.id = ?
  `, [messageId], (err, message) => {
    if (err || !message) {
      return res.status(404).json({ success: false, error: 'Message not found' });
    }

    res.json({ success: true, message: message });
  });
});
```

- router.get('/:id', ...): Đây là route GET với tham số động :id. Ví dụ khi truy cập /api/messages/3, thì req.params.id sẽ nhận giá trị 3.
- requireAuth: Middleware chỉ kiểm tra người dùng đã đăng nhập hay chưa (kiểm tra session).
- req.db.get(...): Thực hiện câu lệnh SQL để lấy thông tin message theo id.
- **Quan trọng nhất**: Câu SQL chỉ có điều kiện WHERE m.id = ?, **không có điều kiện nào liên quan đến user hiện tại** (sender_id hoặc recipient_id phải bằng req.session.userId).

→ Nghĩa là server tin tưởng hoàn toàn vào :id do người dùng gửi lên, không kiểm tra xem người dùng có quyền xem message đó hay không.

Giải thích về kĩ thuật IDOR (Insecure Direct Object Reference)

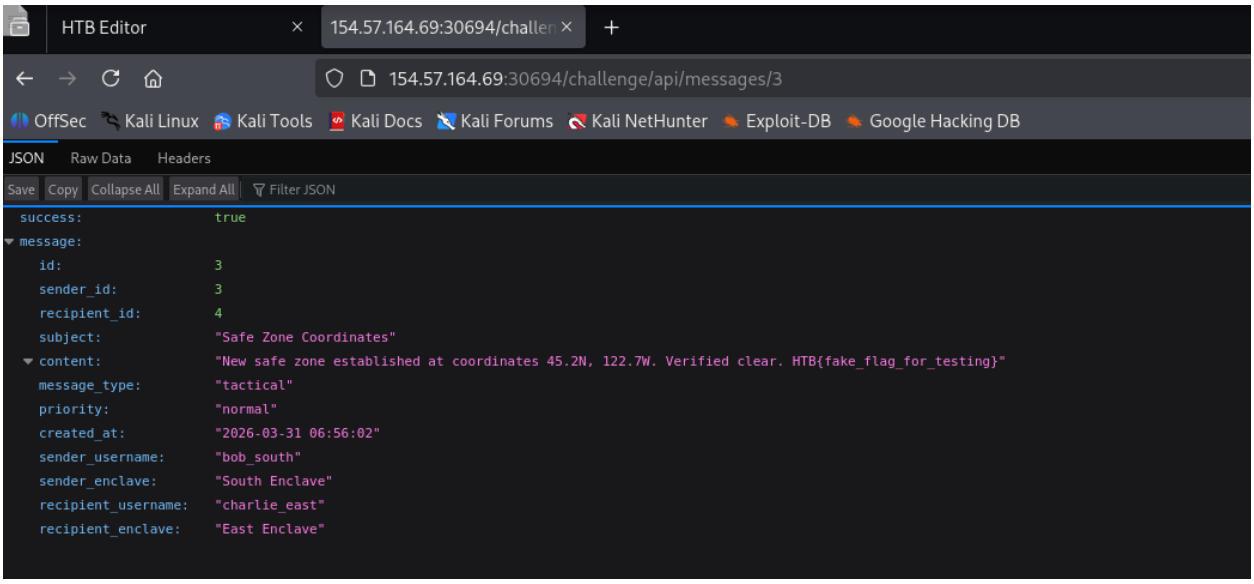
IDOR là lỗ hổng bảo mật xảy ra khi ứng dụng cho phép người dùng truy cập trực tiếp vào một đối tượng (object) thông qua tham số (thường là ID), mà **không kiểm tra quyền sở hữu hoặc quyền truy cập** của người dùng đó.

Ví dụ đơn giản về IDOR:

- Một website ngân hàng có chức năng xem thông tin tài khoản: /account/12345
- User A (tài khoản 12345) có thể xem được thông tin của mình.
- Nhưng nếu User B thử truy cập /account/67890 (ID của User C) và server không kiểm tra quyền → User B sẽ xem được thông tin tài khoản của User C.

Trong Commnet, tình huống tương tự:

- User testuser (user bình thường) không nên đọc được message của bob_south và charlie_east.
- Nhưng chỉ cần thay :id thành 3, user này vẫn đọc được message chứa thông tin quan trọng (và flag).



4. Vá lỗ hỏng

Để khắc phục lỗ hổng IDOR, cần thêm cơ chế kiểm tra quyền sở hữu khi lấy message theo ID. Không thể chỉ dựa vào requireAuth.

Mình đã sửa hàm router.get('/:id', ...) trong file routes/messages.js như sau:

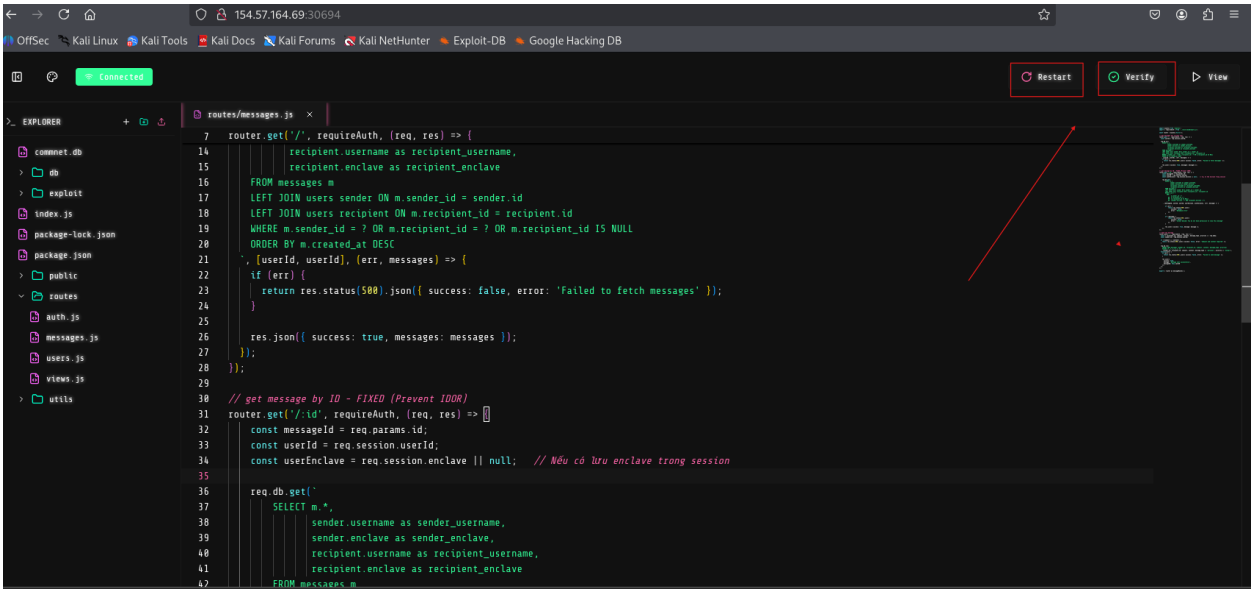
```
// get message by ID - FIXED (Prevent IDOR)
router.get('/:id', requireAuth, (req, res) => {
    const messageId = req.params.id;
    const userId = req.session.userId;
    const userEnclave = req.session.enclave || null;   // Nếu có lưu enclave
trong session

    req.db.get(`
        SELECT m.*,
               sender.username as sender_username,
               sender.enclave as sender_enclave,
               recipient.username as recipient_username,
               recipient.enclave as recipient_enclave
        FROM messages m
```

```
        LEFT JOIN users sender ON m.sender_id = sender.id
        LEFT JOIN users recipient ON m.recipient_id = recipient.id
        WHERE m.id = ?
          AND (
                m.sender_id = ?
            OR  m.recipient_id = ?
            OR  (m.recipient_id IS NULL)
            OR  (sender.enclave = ? AND recipient.enclave = ?)
          )
    `, [messageId, userId, userId, userEnclave, userEnclave], (err, message) => {

        if (err) {
            return res.status(500).json({
                success: false,
                error: 'Database error'
            });
        }

        if (!message) {
            return res.status(403).json({
                success: false,
                error: 'Access denied: You do not have permission to view this
message'
            });
        }

        res.json({ success: true, message: message });
    });
});
```

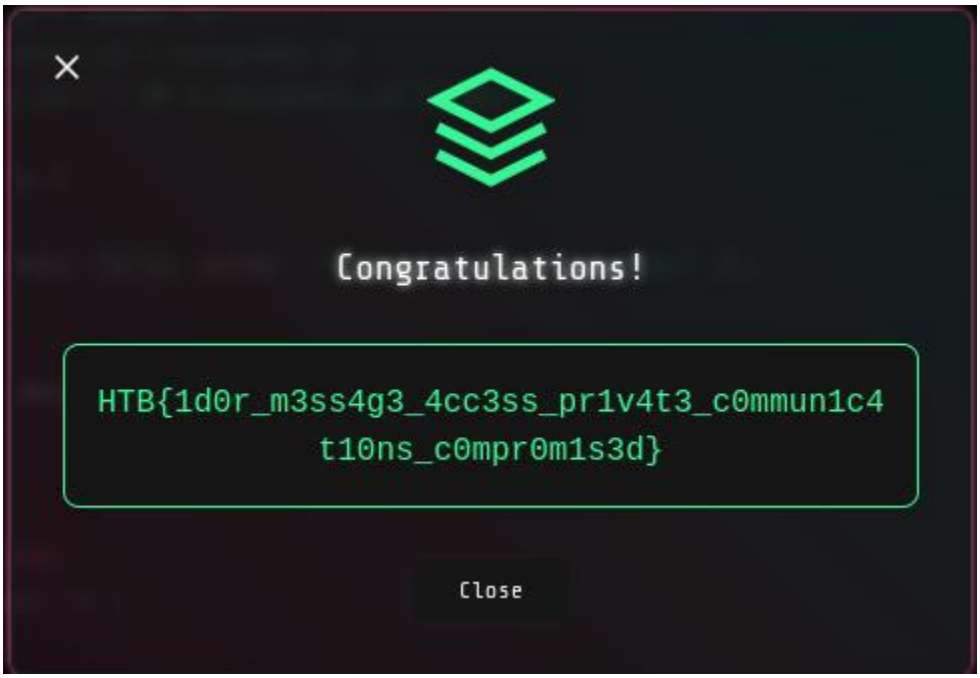
Giải thích cách vá:

1. **Thêm kiểm tra quyền sở hữu**: Kiểm tra message phải do user hiện tại gửi (sender_id = userId) hoặc nhận (recipient_id = userId), hoặc thuộc cùng enclave.
2. **Trả về lỗi 403**: Nếu không có quyền truy cập, thay vì chỉ trả 404 (Message not found).
3. **Sử dụng session**: Lấy userId và userEnclave từ req.session để so sánh.

Ta nhấn Restart và Verify sau khi vá.


154.57.164.69:30694
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Connected
Restart
Verify
View
EXPLORER
commnet.db
db
exploit
index.js
package-lock.json
package.json
public
routes
auth.js
messages.js
users.js
views.js
utils
routes/messages.js
router.get('/', requireAuth, (req, res) => {
recipient.username as recipient_username,
recipient.enclave as recipient_enclave
FROM messages m
LEFT JOIN users sender ON m.sender_id = sender.id
LEFT JOIN users recipient ON m.recipient_id = recipient.id
WHERE m.sender_id = ? OR m.recipient_id = ? OR m.recipient_id IS NULL
ORDER BY m.created_at DESC
`, [userId, userId], (err, messages) => {
if (err) {
return res.status(500).json({ success: false, error: 'Failed to fetch messages' });
}
res.json({ success: true, messages: messages });
});
});
// get message by ID - FIXED (Prevent IDOR)
router.get('/:id', requireAuth, (req, res) =>
const messageId = req.params.id;
const userId = req.session.userId;
const userEnclave = req.session.enclave || null; // Nếu có lưu enclave trong session
req.db.get(`
SELECT m.*,
sender.username as sender_username,
sender.enclave as sender_enclave,
recipient.username as recipient_username,
recipient.enclave as recipient_enclave
FROM messages m

Kết quả


Congratulations!
HTB{1d0r_m3ss4g3_4cc3ss_pr1v4t3_c0mmun1c4t10ns_c0mpr0m1s3d}
Close